

Atividade Prática: Prevendo a Qualidade de Vinhos com Regressão

Objetivo

Você vai usar técnicas de regressão para prever a qualidade de vinhos (nota de 0 a 10) com base em características físico-químicas.

Vamos treinar dois modelos:

1. Regressão Linear
2. Um outro algoritmo de regressão – você poderá escolher entre várias opções disponíveis no Scikit-learn (explicadas adiante).

Depois de treinar e testar os modelos, você deverá:

- Comparar a precisão de cada um.
 - Descobrir quais características (colunas) são mais importantes para determinar a qualidade do vinho (quando o modelo escolhido permitir essa interpretação).
 - Responder qual modelo se saiu melhor e por quê.
-

Conceitos importantes que vamos usar

- Regressão: quando queremos prever um número (a nota do vinho). Diferente da classificação, que prevê uma categoria.
- Overfitting: quando o modelo decora os dados de treino, mas vai mal em dados novos (teste). Vamos ficar de olho nisso.
- Coeficientes (na Regressão Linear): mostram como cada característica influencia a nota. Se o coeficiente é positivo, aumentar aquela característica tende a aumentar a nota; se negativo, tende a diminuir.
- Importância das features (em modelos como Árvore, Random Forest, etc.): alguns modelos calculam quais características foram mais úteis para fazer as previsões.

- Padronização (StandardScaler): técnica para deixar todas as características na mesma escala. É essencial para algoritmos baseados em distâncias (como KNN) e também ajuda a interpretar os coeficientes da Regressão Linear. Nesta base de dados, a padronização não é obrigatória, pois as features estão em unidades diferentes mas as magnitudes não são extremamente díspares. No entanto, aplicá-la é uma boa prática e não prejudica os modelos – pelo contrário, pode melhorar o desempenho de alguns.
 - Métricas de avaliação: vamos usar três:
 - R^2 (coeficiente de determinação): varia de 0 a 1. Quanto mais próximo de 1, melhor o modelo explica os dados.
 - RMSE (Raiz do Erro Quadrático Médio): erro médio das previsões, na mesma unidade da nota. Quanto menor, melhor.
 - MAE (Erro Absoluto Médio): também é um erro médio, mas sem elevar ao quadrado. Mais fácil de interpretar.
-

Dataset

O arquivo `qualidade_vinho.csv` contém amostras de vinho tinto. As colunas são:

Coluna	O que significa
fixed acidity	acidez fixa
volatile acidity	acidez volátil
citric acid	ácido cítrico
residual sugar	açúcar residual
chlorides	cloretos

free sulfur dioxide	dióxido de enxofre livre
total sulfur dioxide	dióxido de enxofre total
density	densidade
pH	pH
sulphates	sulfatos
alcohol	teor alcoólico
quality	nota de qualidade (0 a 10) – é o que queremos prever

Passo a passo

1. Carregar e explorar os dados

Escreva o código para:

- Importar as bibliotecas necessárias (pandas, numpy, etc.).
- Carregar o arquivo `qualidade_vinho.csv` em um DataFrame.
- Exibir as primeiras linhas, informações gerais e estatísticas descritivas.
- Verificar se há valores ausentes.

Código de exemplo (você pode usar como base):

```
python
import pandas as pd
import numpy as np

df = pd.read_csv('qualidade_vinho.csv')
```

```
print(df.head())
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

2. Separar os dados em treino e teste

- Defina `X` como todas as colunas exceto `quality`.
- Defina `y` como a coluna `quality`.
- Divida os dados usando `train_test_split` com `test_size=0.2` e `random_state=42` (para reproducibilidade).

Código:

```
python
from sklearn.model_selection import train_test_split

X = df.drop('quality', axis=1)
y = df['quality']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

3. Padronizar as características (opcional, mas recomendado)

A padronização transforma os dados para que cada feature tenha média 0 e desvio padrão 1. Isso é importante porque:

- Algoritmos que usam distâncias (como KNN) são sensíveis à escala: features com valores grandes dominariam o cálculo da distância.
- Na regressão linear, a padronização permite comparar diretamente a magnitude dos coeficientes: quanto maior o valor absoluto do coeficiente, mais importante é aquela feature (desde que os dados estejam padronizados).
- Modelos regularizados (Ridge, Lasso) também se beneficiam da padronização.

Como fazer:

1. Criar um objeto `StandardScaler`.
2. Ajustar o scaler apenas nos dados de treino com `fit_transform()`.
3. Transformar os dados de teste com `transform()` (usando o mesmo scaler ajustado).

Código:

```
python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Agora você tem `X_train_scaled` e `X_test_scaled` para usar nos modelos. Lembre-se: Se você escolher um modelo que não exige padronização (ex.: árvore de decisão), pode usar os dados originais ou os padronizados – ambos funcionarão. Para manter a consistência, usaremos os dados padronizados em todos os modelos.

4. Modelo 1 – Regressão Linear

Agora é com você! Escreva o código para:

- Importar `LinearRegression` do `sklearn.linear_model`.
- Criar e treinar o modelo usando `X_train_scaled` e `y_train`.
- Fazer previsões sobre `X_test_scaled`.
- Calcular as métricas: R^2 , RMSE e MAE. Para o RMSE, lembre-se de usar `np.sqrt(mean_squared_error(...))`.
- Exibir os coeficientes do modelo e o intercepto.
- Criar um DataFrame com os nomes das features e seus coeficientes, ordenado do mais positivo ao mais negativo, para analisar quais características mais influenciam a qualidade.

Dica: Os coeficientes estão no atributo `lr.coef_` e o intercepto em `lr.intercept_`.

5. Modelo 2 – Escolha um outro algoritmo de regressão

Agora você deve escolher um outro modelo de regressão disponível no Scikit-learn. Abaixo estão algumas opções com uma breve explicação:

Modelo	Classe no sklearn	Breve descrição
KNN para regressão	<code>KNeighborsRegressor</code>	Baseia-se nos K vizinhos mais próximos; a previsão é a média das notas dos vizinhos.
Árvore de Decisão	<code>DecisionTreeRegressor</code>	Cria regras de decisão em forma de árvore; previsão é a média dos exemplos em cada folha.
Ridge (Regressão Linear com regularização)	<code>Ridge</code>	Versão da regressão linear que evita overfitting, penalizando coeficientes muito grandes.
Lasso (Regressão Linear com regularização)	<code>Lasso</code>	Similar ao Ridge, mas pode zerar coeficientes de características pouco importantes (seleção automática).

SVR (Support Vector Regression)	SVR	Busca encontrar uma função que se desvie das observações por no máximo um certo valor; pode usar diferentes kernels (linear, polinomial, RBF).
Random Forest	RandomForestRegressor	Conjunto de várias árvores de decisão, cada uma treinada em uma amostra diferente dos dados; a previsão final é a média das previsões de todas as árvores.
Gradient Boosting	GradientBoostingRegressor	Constrói árvores sequencialmente, cada uma tentando corrigir os erros da anterior; costuma ser bem preciso, mas pode overfit se não regularizar.
MLPRegressor (rede neural simples)	MLPRegressor	Rede neural de múltiplas camadas; capaz de aprender relações complexas, mas requer

ajuste de
hiperparâmetros.

Escolha um modelo que você tenha curiosidade em testar. Em seguida, escreva o código para:

- Importar a classe correspondente.
- Criar o modelo (se quiser, altere algum parâmetro – por exemplo, o número de vizinhos no KNN, a profundidade da árvore, etc.).
- Treinar o modelo com `X_train_scaled` e `y_train`.
- Fazer previsões sobre `X_test_scaled`.
- Calcular as mesmas métricas: R^2 , RMSE, MAE.
- Se o modelo escolhido possuir o atributo `feature_importances_` (como árvores, random forest, gradient boosting), obtenha e exiba a importância de cada feature em um DataFrame ordenado.
- Caso contrário (ex.: KNN, SVR), apenas comente que o modelo não fornece essa informação.

Lembre-se de documentar seu código com comentários explicando cada passo.

6. Comparar os modelos

Crie uma tabela resumo com os resultados dos dois modelos (você pode exibir no próprio notebook usando markdown ou simplesmente printar os valores comparados). Por exemplo:

Modelo	R^2	RMSE	MAE
Regressão Linear	...	...	...
[Nome do seu modelo]	...	...	...

Agora responda às seguintes perguntas em células Markdown:

1. Qual modelo apresentou o melhor R^2 ? E o menor erro (RMSE/MAE)?
 2. Considerando as métricas, qual modelo você recomendaria para este problema? Justifique.
 3. Quais são as 3 características mais importantes para definir a qualidade do vinho, de acordo com o melhor modelo?
 - Se o melhor modelo foi a Regressão Linear, olhe os coeficientes com maior valor absoluto.
 - Se foi um modelo que fornece `feature_importances_`, use essa informação.
 - Se foi um modelo que não dá importância (ex.: KNN, SVR com kernel não linear), comente essa limitação e, se desejar, compare com os coeficientes da Regressão Linear como análise complementar.
 4. Os resultados dos dois modelos foram muito diferentes? O que isso pode indicar sobre a relação entre as variáveis e a qualidade do vinho? (ex.: se a Regressão Linear foi muito pior, talvez a relação não seja linear; se a árvore overfitou, pode ter ido mal no teste, etc.)
-

7. Entrega

- Um notebook (`.ipynb`) ou script Python bem comentado, contendo todas as etapas, os resultados e as respostas às perguntas.
 - Nome do arquivo: `seunome_vinho_regressao.ipynb`
-

Dicas importantes

- Use `random_state=42` em todos os lugares que aceitarem esse parâmetro (incluindo os modelos que possuem `random_state`). Isso garante reproducibilidade.
- Consulte a documentação do Scikit-learn sempre que precisar:
<https://scikit-learn.org/>

- Não se preocupe se os resultados não forem excelentes – prever a qualidade do vinho com precisão é desafiador. O foco é aprender a comparar modelos e interpretar os resultados.
- Comente seu código – isso ajuda na correção e no seu próprio entendimento.

Bom trabalho! 🍷📊